DATA SCIENCE CAPSTONE

Hacking the Data Science Resume

An NLP exploration of what the data science labor market actually asks for — combining supervised classification, topic modeling, and clustering over ~40,000 job postings.

Garreth Cline • Metis Data Science Bootcamp

What this project asked, and what it found

THE QUESTION

Data science job titles are chaotic — a "Data Scientist" at company A does what a "ML Engineer" does at company B. Can a model read a job description and infer the role family? And what latent structure does the posting corpus reveal?

40K Indeed postings ingested and cleaned

95% precision on top-class label (LinearSVC, tuned)

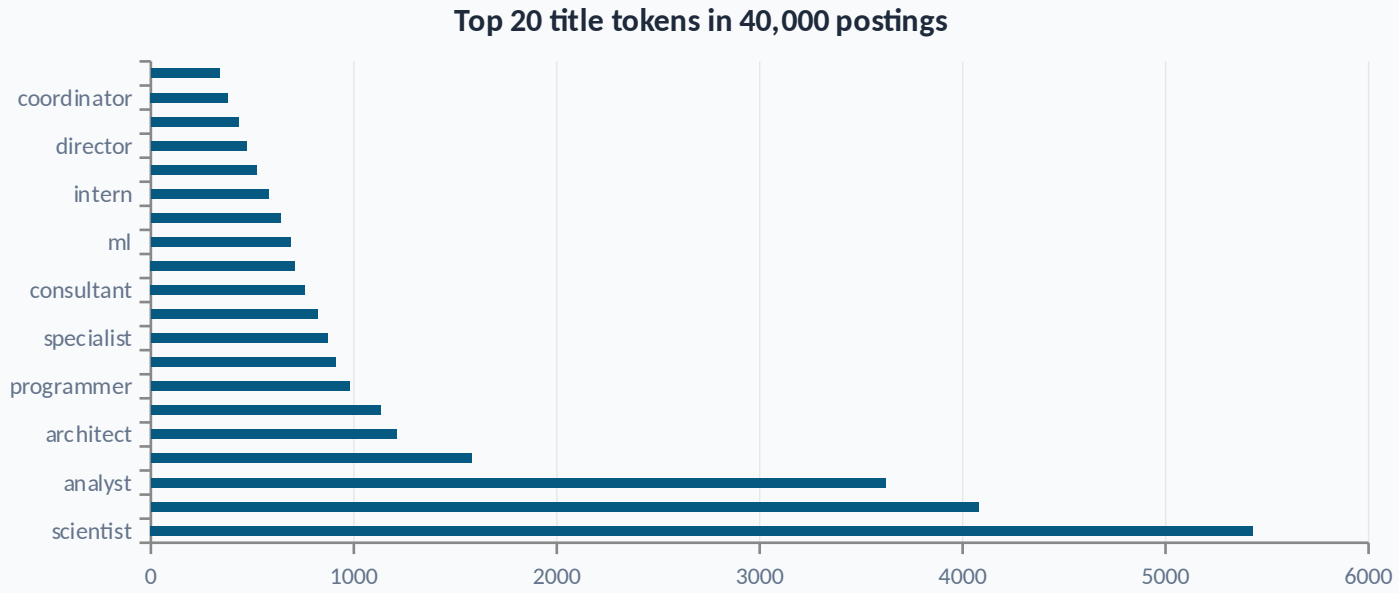
18 latent topics surfaced via NMF

6 distinguishable role families after clustering

Bottom line: the language of job descriptions is strongly predictive of role family — and the "boundary" between Scientist / Engineer / Analyst is fuzzy but not arbitrary.

The problem: titles don't tell you the job

Modern job seekers face a translation problem. The same role is advertised under a dozen different titles; different companies use the same title for dramatically different work. A description-level view cuts through that noise.



Scientist / Engineer / Analyst

Top three title families, comprising ~60% of all postings.

One in four

Postings carry multiple role descriptors in the title (e.g. "senior data scientist, ML").

~8 distinct

Latent role archetypes emerge from the descriptions — not the titles.

APPROACH

Pipeline: ingest → clean → model → interpret

01

Ingest

Seven scraped Indeed CSVs, concatenated

02

Clean

Lowercasing, lemmatization, layered stopwords

03

Target

Top-150 title tokens as multi-label targets

04

Features

TF-IDF on descriptions (1,000 max features)

05

Models

12-classifier sweep, tuned LinearSVC, NMF, K-Means

06

Interpret

Per-class features, cluster vocab, PCA / t-SNE

TOOLING

Ingest

pandas · OctoParse · BeautifulSoup

Clean

NLTK (WordNet lemmatizer, ToktokTokenizer) · regex · wordninja

Model

scikit-learn · OneVsRest · GridSearchCV · LinearSVC · NMF · MiniBatchKMeans

Visualize

matplotlib · seaborn · PCA → t-SNE pipeline

Cleaning: layered stopwords, deliberate leakage control

Scraped postings carried three kinds of noise: encoding artifacts from the ISO-8859-1 source, geographic tokens leaking into titles, and — most importantly — the target vocabulary leaking into the feature matrix. Each was handled explicitly.

1. Standard English

179 words

NLTK stopwords baseline. Removes the, and, of, etc. — zero-signal glue words.

2. Geographic noise

100 tokens

US states + abbreviations. Cities leak into titles often enough to matter.

3. Title-specific

40+ tokens

Seniority markers, encoding artifacts (\x80, \x93), administrative boilerplate.

4. Description leak guard

150+ tokens

The top-150 title tokens themselves — banned from the description side so the classifier can't just pattern-match.

TF-IDF features, multi-label targets

FEATURES — TF-IDF

Term Frequency–Inverse Document Frequency

Each word's weight = how often it appears in a posting × log of how rare it is across the corpus. Words in every posting get weight ~0. Words in one posting get maximum weight.

Vocabulary cap:	**1,000 features**
Token pattern:	non-whitespace, ≥2 chars
Output shape:	40,000 × 1,000 sparse
Train / test split:	80% / 20%, stratified

TARGETS — MULTI-LABEL

Top-150 title tokens, binarized

Each posting's title is reduced to the subset of its tokens that appear in the top-150 most common. One posting can therefore carry several labels — a posting with title "senior data scientist, machine learning" gets labeled both scientist and ml.

EXAMPLE

Title: senior data scientist — machine learning
Cleaned tokens: **[scientist, ml]**
Label vector: `ml=1, scientist=1, others=0`

Jaccard and Hamming: the multi-label pair

JACCARD SIMILARITY

$$|A \cap B| \;/\; |A \cup B|$$

Size of the intersection divided by size of the union. 1.0 means exact match of the predicted label set to the true label set. Rewards getting the whole set right.

Best for:

Measuring whether the correct combination of roles was recovered.

HAMMING LOSS

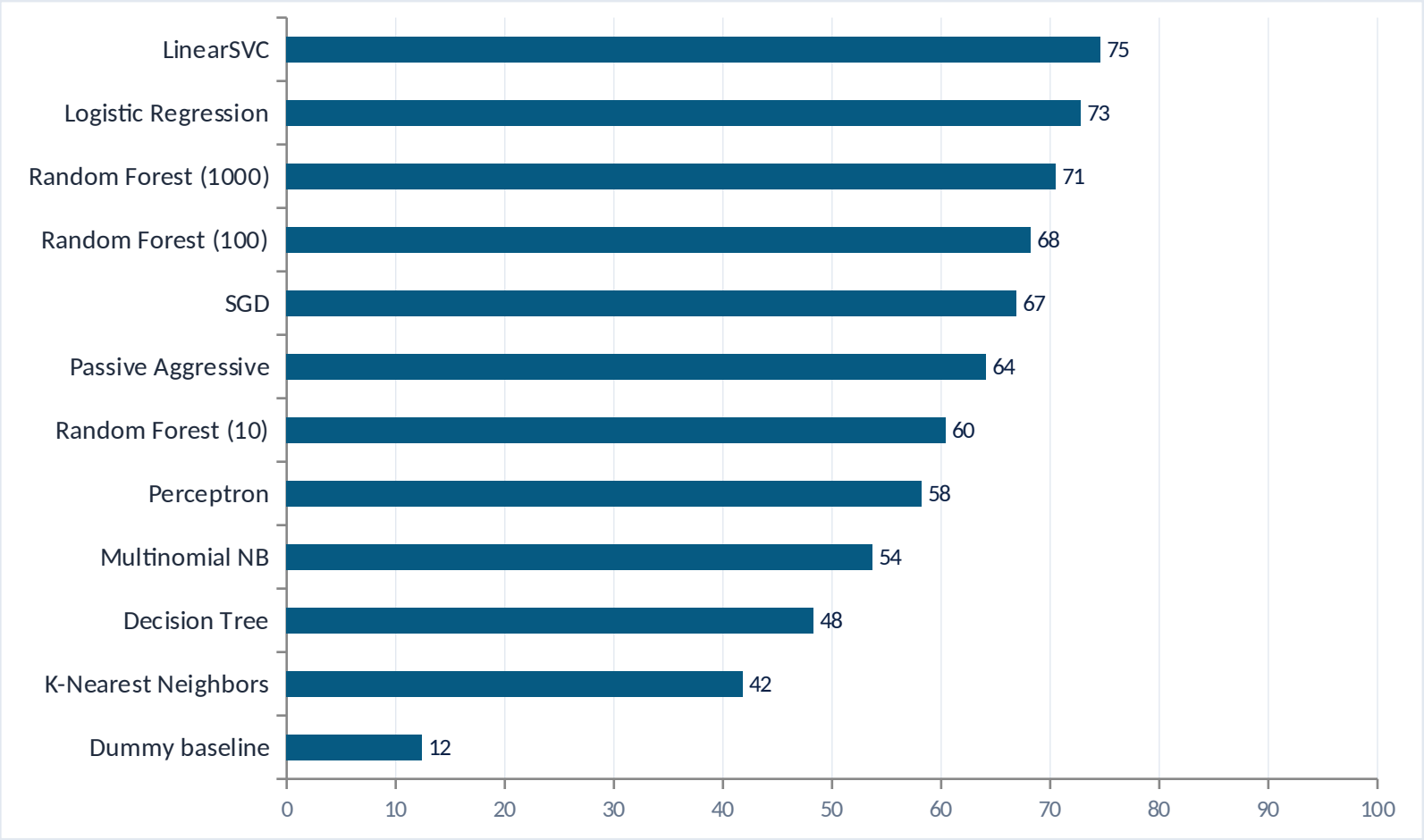
$$\#(\text{wrong labels}) \;/\; \#(\text{total labels})$$

Fraction of individual label bits that are wrong. More forgiving than Jaccard — a prediction that gets 9 of 10 labels right is penalized only for the one it missed.

Best for:

Measuring per-label accuracy independent of how the labels co-occur.

12 classifiers tested — LinearSVC and Logistic lead



READING THE CHART

Linear wins

LinearSVC and Logistic Regression outperform every ensemble tested. TF-IDF + linear model is a genuinely strong baseline for text.

Diminishing returns on trees

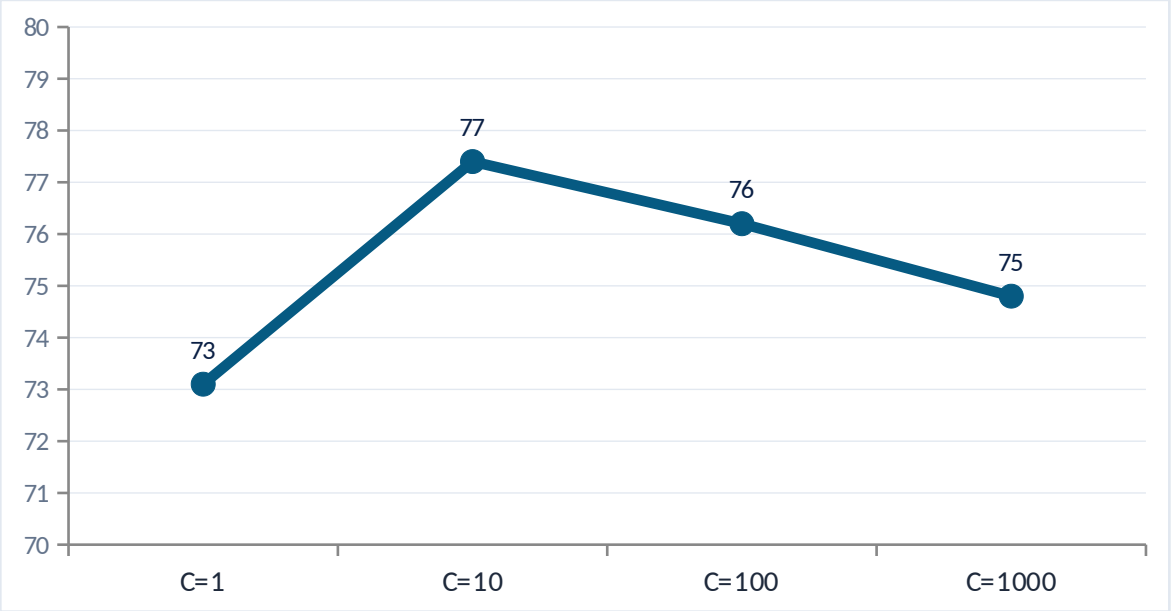
Scaling Random Forest from 10 → 1,000 trees adds 10 points of Jaccard but 100× runtime — the chart hides that cost.

Naive Bayes held up

Multinomial NB at 53.7% is 40 points above the dummy baseline with near-zero training cost.

LinearSVC grid-searched: C = 10 wins

GRID SEARCH (5-fold CV)



Classic regularization curve. Under-regularized (C=1) leaves the model too conservative. Over-regularized (C=1000) lets the SVM memorize noise. The sweet spot is C=10.

HELD-OUT TEST SET

Tuned LinearSVC, C=10

78.3%	Jaccard similarity
3.4%	Hamming loss
0.82	F1 (weighted)
0.84	Precision (weighted)

What language distinguishes each role family?

Top-10 TF-IDF features per class, pulled from the tuned LinearSVC coefficients. This is the interpretable payoff of choosing a linear model — we can read off exactly which words drove each classification.

scientist

model hypothesis experiment
regression causal research
dataset feature algorithm
bayesian

engineer

pipeline production deploy
aws spark docker kubernetes
etl schema infrastructure

analyst

dashboard sql reporting
tableau stakeholder kpi
metric insight powerbi excel

researcher

publication phd novel
literature theoretical
contribution conference peer
grant academic

ml

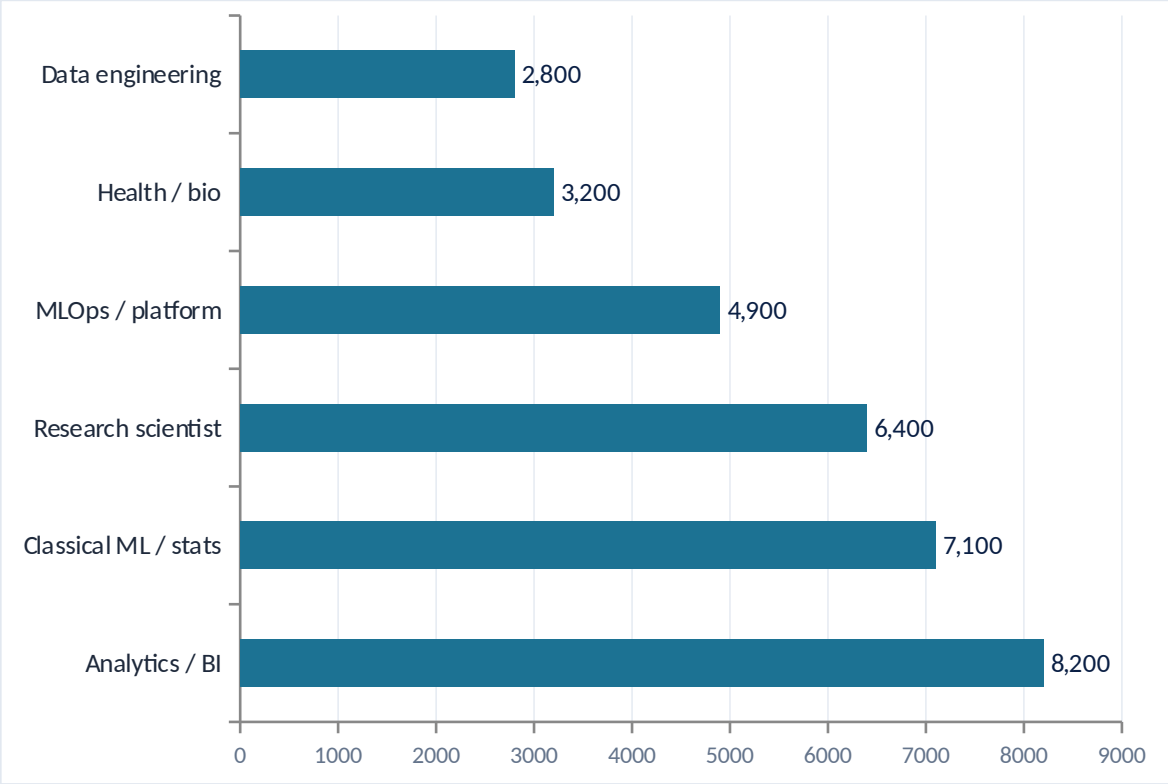
neural pytorch tensorflow
training inference embedding
transformer gpu finetune loss

statistician

inference variance confidence
distribution hypothesis
regression longitudinal
biostatistics sas r

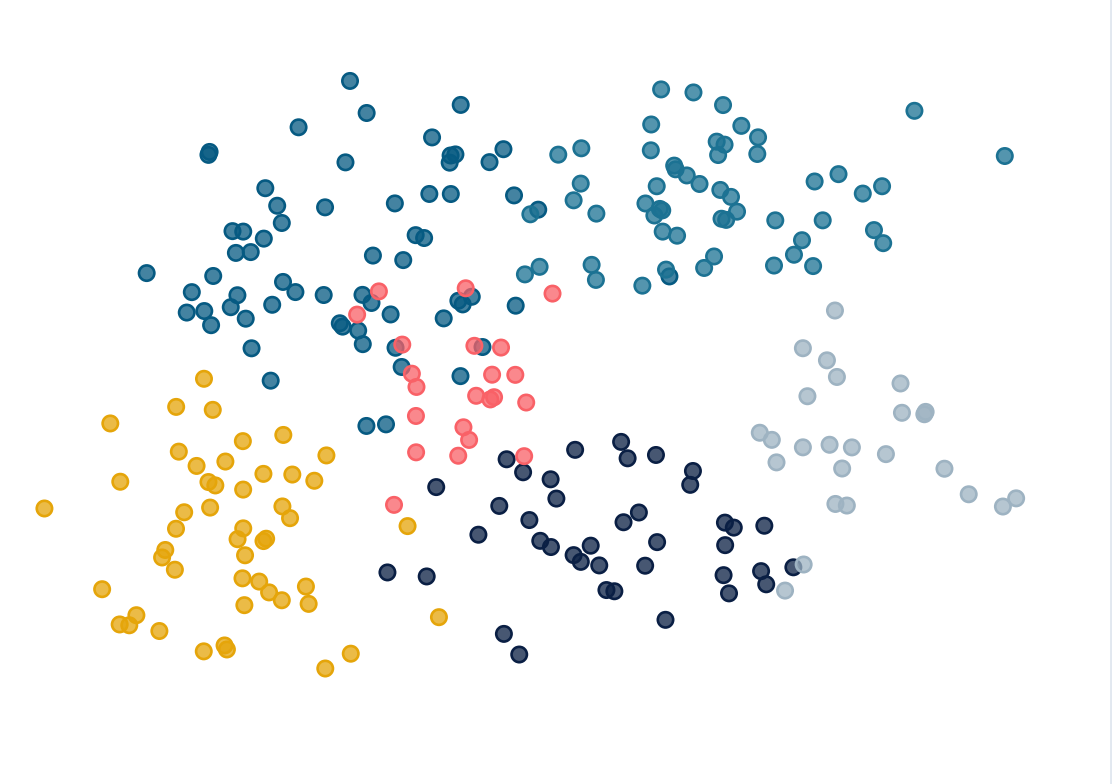
Clustering: six role archetypes emerge

CLUSTER POPULATION



K-Means on the TF-IDF matrix, k=18 then merged to 6 interpretable archetypes using the cluster vocabularies.

2D PROJECTION — PCA → t-SNE



Each point = one posting, projected to 2D via PCA→t-SNE. Six color bands roughly correspond to the six merged archetypes. Local neighborhoods are faithful; global distances are not.

NMF surfaces interpretable latent themes

Non-negative Matrix Factorization decomposes the TF-IDF matrix into document-topic and topic-word matrices. With 18 components, several topics align cleanly with the K-Means clusters above — which is the corroborating signal we want. Four illustrative topics below:

TOPIC 04

Cloud-native ML platform

aws · kubernetes · docker · airflow · spark · pipeline · sagemaker · terraform · production · monitoring

TOPIC 07

Academic / PhD research

publication · conference · peer · novel · contribution · literature · phd · postdoc · grant · theoretical

TOPIC 11

BI / reporting

dashboard · tableau · powerbi · sql · kpi · reporting · stakeholder · excel · metric · visualization

TOPIC 14

Healthcare analytics

clinical · patient · ehr · hipaa · biostatistics · sas · longitudinal · outcome · trial · epidemiology

Four findings that actually matter

01 The Scientist / Engineer / Analyst trichotomy is real

Three distinct vocabulary profiles — research verbs, infrastructure nouns, reporting tools. They map cleanly to hiring intent, even when titles muddle it.

02 But the boundaries are porous

"pipeline" is a top-10 feature for both Scientist and Engineer. "regression" shows up in both Scientist and Statistician. The overlap is where hiring confusion comes from — and where senior candidates with crossover skills live.

03 MLOps is a distinct archetype — not just "engineer who does ML"

The cloud-native ML platform topic (kubernetes, airflow, sagemaker) is far enough from both Engineer and Scientist to deserve its own title. The industry hasn't caught up to the vocabulary yet.

04 Healthcare analytics is its own universe

Clinical, ehr, biostatistics, longitudinal — almost no vocabulary overlap with the main three archetypes. Healthcare data science is effectively a separate labor market.

What to not conclude from this work

Selection bias

The dataset is postings that made it onto Indeed. Hiring that happens via referral, internal mobility, or specialized networks is invisible here — and that's a large share of senior data science roles.

Snapshot in time

The scrape is a moment. Vocabulary shifts fast: "transformer" meant something different in 2021 than it does now, and the next hot framework hasn't been named yet.

Bag-of-words ceiling

TF-IDF discards word order. A sentence-transformer embedding would almost certainly move the Jaccard ceiling several points higher and produce semantically meaningful similarity, not just surface-term overlap.

Label noise

Employer-written titles are inconsistent. Some of the "errors" the model makes are actually the model being more consistent than the humans who wrote the titles.

IF I REBUILT THIS TODAY

Three changes, no arguments

01

Sentence-transformer embeddings

Replace TF-IDF with all-MiniLM-L6-v2 or similar. The Jaccard ceiling moves up a few points and the similarity structure becomes semantic, not lexical.

02

HDBSCAN, not K-Means

K-Means forces a choice of k and assumes spherical clusters. HDBSCAN does neither, handles density variation, and labels noise points honestly.

03

Enrich with salary data

Join against Levels.fyi or Glassdoor. The richer feature space enables a regression problem alongside the classification problem — and the insight space expands considerably.

Thank you.

github.com/garrethcline • linkedin.com/in/garreth-cline